PORTAL WALKTHROUGH

Dhanalakshmi Jewellery

A walk through the store - from first click to fulfilment

This is the project as a visitor, a customer, and a store administrator would experience it. The focus is not just on what each screen shows, but on what happens behind it.

EDITION	DATE	BUILT WITH
Version 1.1	14 October 2026	React, Spring Boot, MySQL

02

THE SHAPE OF THE PORTAL

Think of it as three rooms

The public storefront is the front window. The customer account is the fitting room. The administrator dashboard is the counter where orders and stock are handled.

01 · STOREFRONT

Open to everyone

Home, collection, product details, search, filters, sorting, and the local cart.

02 · MY ACCOUNT

After sign-in

Checkout, profile details, and the signed-in customer's own order history.

03 · BACK OFFICE

For administrators

Order monitoring, status changes, and catalogue management.

UNDER THE COUNTER

One request, three layers

React + Vite
pages and forms

—

Spring Boot
API and security

—

MySQL
users, stock, orders

Email is a side service, not the centre of the app. Gmail SMTP is used for registration and password-reset one-time passwords (OTPs).

WHO CAN DO WHAT

Access at a glance

Area	Visitor	Customer	Administrator
Browse products	Yes	Yes	Yes
Build a cart	Yes, in the browser	Yes	Yes
Checkout and view orders	Sign-in required	Own orders only	May use customer view
Manage orders and products	No	No	Yes

The important distinction: React decides which screen to show. Spring Security decides whether the request is allowed. Hiding an Admin link is useful; rejecting a customer's admin API request is the real protection.

FIRST VISIT

From email address to a verified account

1 Create the account

At `/register`, the customer enters a name, email address, and strong password. Duplicate email addresses are rejected.

2 Check the inbox

The portal moves to `/verify-otp`. A six-digit OTP confirms that the email belongs to the person registering. Resend is available after a 60-second cooldown.

3 Sign in

A successful login returns a JSON Web Token (JWT). The app then asks the server for the account's current role.

4 Land in the right place

Customers continue to `/customer`; administrators land on `/admin`. Administrators can still open the customer side when they want to view the store as a shopper.

A password the backend accepts

At least eight characters, including uppercase, lowercase, a number, and an accepted special character.

> The browser remembers the session. The database remains the source of truth.

Stored locally: token, email, role, and cart. The role helps navigation, but changing it in browser storage does not create admin access.

IF THE PASSWORD IS FORGOTTEN

A short recovery loop

`/forgot-password` — **Email OTP** — `/reset-password` — **Back to login**

A useful failure message: if the frontend cannot reach the backend, say that clearly. “Invalid email or password” should be reserved for an actual authentication failure.

A successful login proves the full chain: the account is verified, BCrypt accepts the password, the server can sign a token, and the frontend can continue with an authenticated request.

THE SHOP FLOOR

Browse first. Sign in only when it matters.

A visitor should be able to explore the collection and build a cart without being interrupted. Authentication arrives at checkout, when the action becomes personal.

DISCOVER

Home and collection

Category tiles lead into filtered results. On `/jewellery`, visitors can search by name, description, or category, then sort by featured order, price, or name.

CONSIDER

Product details

At `/product/:id`, the shopper sees price, description, category, stock, and related items. An out-of-stock piece cannot be added to the cart.

COLLECT

The local cart

Quantity can move up or down, but never above known stock. Removing the last unit removes the line. Totals update immediately in React.

COMMIT

Checkout

An empty cart returns to `/cart`. A real checkout requires a valid session, a delivery address, and a ten-digit phone number.

WHAT HAPPENS ON PLACE ORDER

The server checks the cart again

Validate customer — **Read live product** — **Check stock** — **Create order** — **Reduce stock**

Why the second check? Browser totals are previews. The backend reads the current database price, confirms stock, calculates the total, stores the order, and updates inventory.

After success: the cart is cleared and the customer dashboard opens. New orders begin as `PLACED`.

Current order model: a cart with several products creates several order records - one per cart item. There is no combined order header or payment transaction yet.

05

AFTER CHECKOUT

The account becomes the customer's receipt drawer

At `/customer`, the signed-in person sees profile details and only their own orders. The page is deliberately useful to administrators too, so they can browse the customer experience without changing accounts.

Profile

Name, email address, account type, and verification status come from `/api/customer/profile`.

Order history

Orders are shown newest first, with product, quantity, total, status, and date.

Back to the store

Continue Shopping returns the customer to the collection, including from an empty-order state.

WHAT THE CUSTOMER CAN READ

One row, one piece, one current status

Field	What it tells the customer
Order ID	A reference for the stored order record
Product and quantity	The piece and number ordered
Total	The database price multiplied by quantity
Status	Where fulfilment currently stands
Order date	When the order record was created

> The customer sees their order. The administrator sees the queue behind it.

Logout is a clean handoff. It removes token, email, role, and cart from the browser before returning to the public experience.

ADMINISTRATOR VIEW

Orders first, catalogue second

The order queue

The dashboard collects total orders, gross recorded order value, placed orders, and delivered orders above a detailed table. Each row includes customer, product, quantity, total, phone, status, and order date.

PLACED · CONFIRMED · SHIPPED · DELIVERED · CANCELLED

Status changes are saved first, shown second. The row updates only after `PUT /api/admin/orders/{id}/status` succeeds. Any value outside the five listed states is rejected.

Before changing a status

- Match the order ID and customer.
- Avoid skipping fulfilment stages.
- Remember that cancellation does not restore stock automatically.
- Treat the revenue card as gross order value, not settled payment revenue.

CATALOGUE WORK

Add, edit, or retire a product

1 **Open** `/admin/products`

The route is protected for administrators. Until a dashboard link is added, open it directly.

2 **Enter the details**

Name, price, and stock are required. Description, category, and image URL are optional.

3 **Save with validation**

Price and stock cannot be negative. Price allows two decimal places and up to twelve digits of precision.

Stock cues

0: unavailable
1-3: low-stock warning
4+: available

Images: the form stores a URL; it does not upload files. A public version should use controlled, durable image hosting.

Use delete carefully. A temporary stock-out should usually be stock = 0, not deletion. Historical order references may still depend on the product record.

THE PART WORTH EXPLAINING IN AN INTERVIEW

The UI guides. The backend guards.

A customer can edit browser storage, guess a route, or call an endpoint directly. None of those should turn them into an administrator.

Browser	The route guard checks for a token. A customer opening `/admin` is redirected to `/customer`. This is navigation, not the final security decision.
API client	Requests carry `Authorization: Bearer <token>`. A `401` clears the local session and cart.
JWT filter	The server verifies the signature, reads the email, and reloads the account from MySQL.
Spring Security	`/api/admin/**` requires `ROLE_ADMIN`. Product reads and account flows remain public where intended.

FOUR PRACTICAL SAFEGUARDS

Small decisions that hold the line

BCrypt passwords
Stored passwords are hashed and are never returned through the profile response.

Verified accounts
An account remains disabled in Spring Security until its email OTP is accepted.

Signed tokens
`JWT_SECRET` must provide at least 32 bytes for HS256 signing. A shorter key causes token generation to fail.

Server-loaded roles
Changing `userRole` in local storage may change a visible link, but the server still reads the role from the database.

Never document live secrets. Passwords, active JWTs, database credentials, mail app passwords, and signing keys belong in environment variables - not screenshots, commits, or a portfolio PDF.

The clean test: use a customer token against an admin endpoint. The expected result is `403`. Repeat with an administrator token and the same endpoint should succeed.

A SIMPLE PORTFOLIO DEMO

Tell one customer story, then reveal the machinery

Customer story - about six minutes

1. Start on Home and enter the collection through a category.
2. Search, filter, and sort. Open an in-stock product.
3. Add it to Cart and change the quantity.
4. Sign in with a verified test customer.
5. Complete checkout with test delivery details.
6. Open My Account and show the new order.

Admin story - about four minutes

1. Sign out and log in with an administrator test account.
2. Find the order and move it from `PLACED` to `CONFIRMED`.
3. Open `/admin/products` and show add/edit/delete.
4. Finish with the security point: a customer token receives `403` from an admin API.

BEFORE ANYONE CLICKS THE LINK

Five quiet checks prevent an awkward demo

- ✓ The backend is running and `/api/products` responds.
- ✓ MySQL contains at least one in-stock product.
- ✓ SMTP is configured if registration or reset will be shown.
- ✓ The frontend points to the deployed backend URL.
- ✓ Verified customer and admin test accounts exist.
- ✓ No secret or active token is visible in a terminal.

> **End on the strongest point:** the same customer credentials that work for shopping are rejected by the administrator API. That makes the security story visible, not theoretical.

PUBLIC-DEMO REALITY CHECK

What is ready, and what still needs work

Area	Current position	Before a public demo
Backend hosting	Render blueprint is present	Set the required environment variables
Frontend API	Most calls use `VITE_API_URL`	Remove any remaining localhost-only request
CORS	Global origin is configurable	Review controller-level localhost annotations
Email	Gmail SMTP is integrated	Use an app password securely
Payments	No gateway is implemented	Call checkout “order placement”

WHEN SOMETHING GOES WRONG

Start with the failing layer

No server response

Confirm the backend is running and the frontend points to it. Local development expects port `8080`.

Login fails

Check the account, email verification, password, and a `JWT_SECRET` of at least 32 bytes.

No OTP email

Check mail username, mail app password, Gmail SMTP access, and the resend cooldown.

Checkout fails

Check the session, live stock, a non-empty address, and a ten-digit phone number.

USEFUL ROUTES TO KEEP NEARBY

The short route list

/jewellery	Searchable and sortable collection
/cart · /checkout	Cart and authenticated order placement
/customer	Profile and the customer's own orders
/admin	Administrator order queue
/admin/products	Administrator catalogue management

> **A truthful project line:** “A full-stack jewellery commerce portal with verified accounts, JWT authentication, stock-aware ordering, server-side role authorization, and administrator operations.” Add “deployed” or “online payments” only after those parts are real.